

An overview of 3D Vision-Language Models

Márcus Lobo and Vitor Matias

Afonso Paiva, Jeová Farias, Tiago Novello, Moacir Ponti

Tutorial Page



Bowdoin



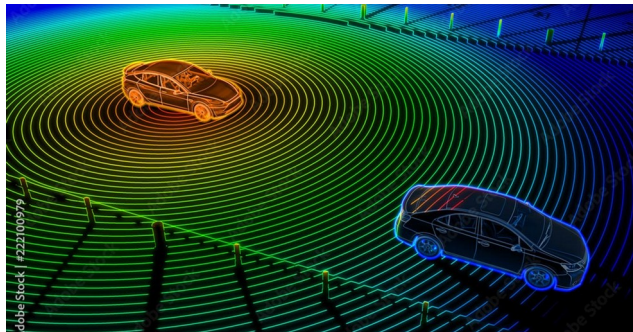
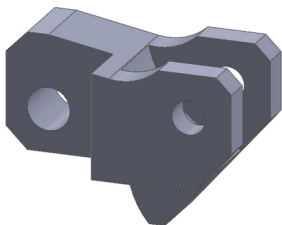
Goals & Motivation

Goals & Motivation

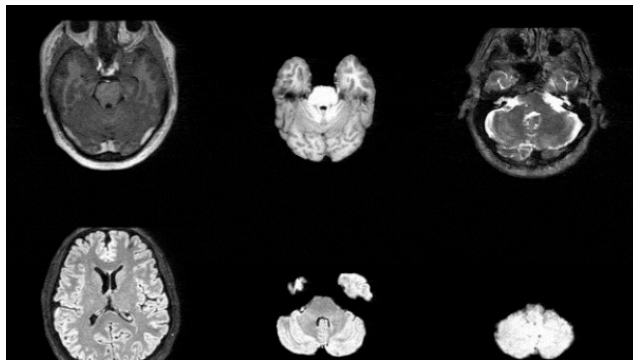


Robotics

Engineering



LiDAR



Medical

Many Others:

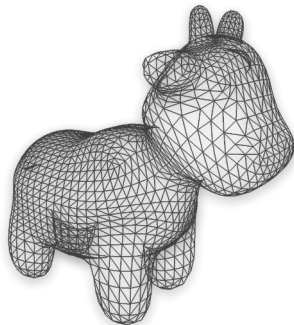
- “Google images”
for 3D objects

How can I represent the data?

Point cloud



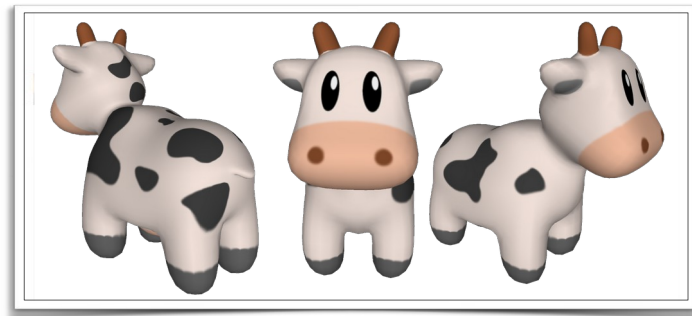
Mesh



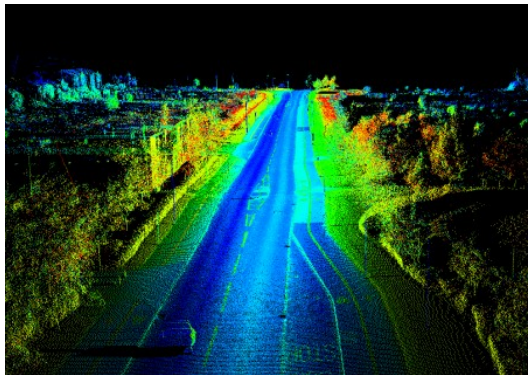
3DGS



Multi-view



Point clouds: various sensors



Lidar



Structure from motion



Kinect and depth cameras

- What are the tasks?

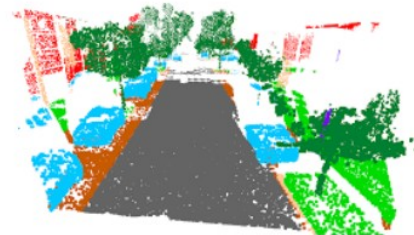


Gaussian Splatting

Traditional closed-set 3D tasks



It is a chair!



Correspondence

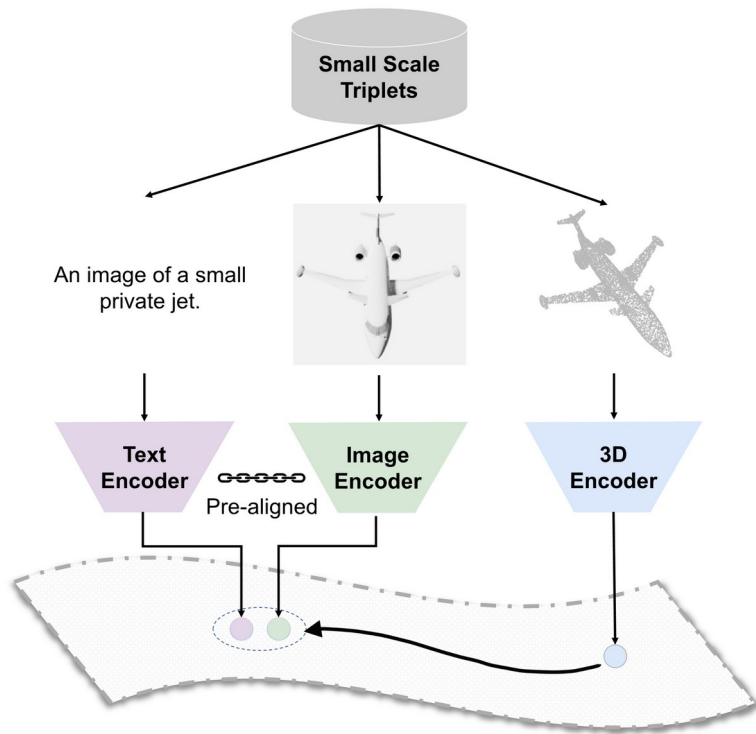


Goals & Motivation

- We are moving *beyond* traditional closed-set 3D recognition
- To a more broad generalization of tasks

Problem Statement

- Given a set of triplets:
 - 3D representation, Image and Text
- We will alignment these multimodal representations
- **3D Vision-Language Models (3D VLMs)**



Source: ULIP framework.

A goal for alignment



A goal for alignment



Impact

Great companies are building on 3D + Language

Meta

SAM 3D

Single image → 3D
geometry, texture
and layout

Microsoft

TRELLIS

Text / image → 3D
meshes, radiance
fields, Gaussians

Google DeepMind

RT-2

Vision-language-action
model for robot control

NVIDIA

GR00T N1

Foundation model
for humanoid
robots

Salesforce

ULIP / ULIP-2

3D-image-text
contrastive
alignment

Alibaba

Qwen3-VL

VLM backbone used by
3D VLLMs

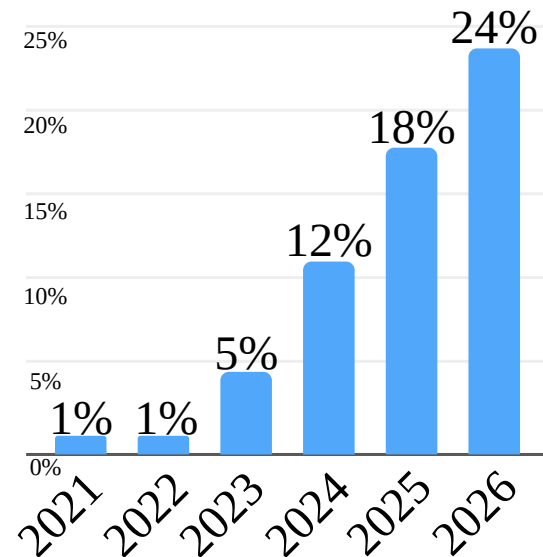
Impact

VLMs are already everyday products

Google Lens in Chrome: select anything on a page and ask about it.



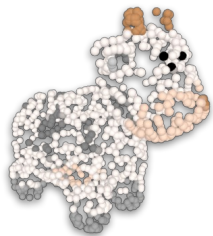
CVPR N° of VLM Papers



How can I represent the data?

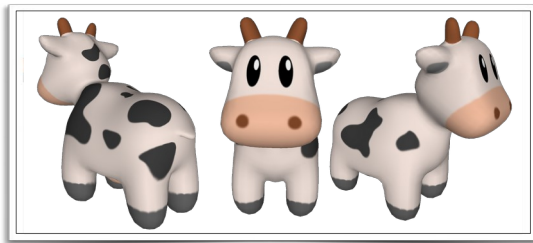
- For 3D VLMs we will focus on the main 3D representations:

Point cloud



$$\mathcal{S} = \{p_i\}_{i=1}^N, \quad p_i \in \mathbb{R}^3$$

Multi-view



$$\mathcal{S} = \{v^{(k)}\}_{k=1}^V, \quad v^{(k)} \in \mathbb{R}^{H \times W \times 3}$$

3DGS



$$\mathcal{S} = \{(\mu_i, \Sigma_i, \sigma_i, c_i)\}_{i=1}^M$$

- And text

Where does the language come from?

Training data: A **shape** or **scene** is paired with language that describes it.

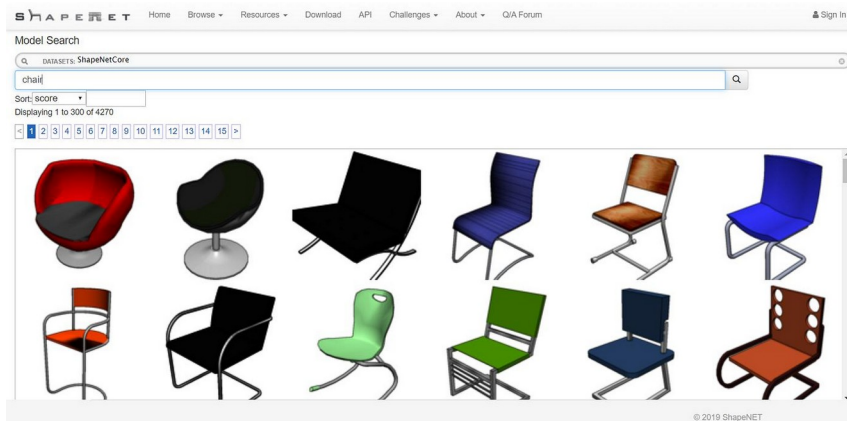
Text encoder: CLIP maps the phrase into a semantic embedding.

3D encoder: The 3D representation learns to approach its matched text.



Datasets

ShapeNet



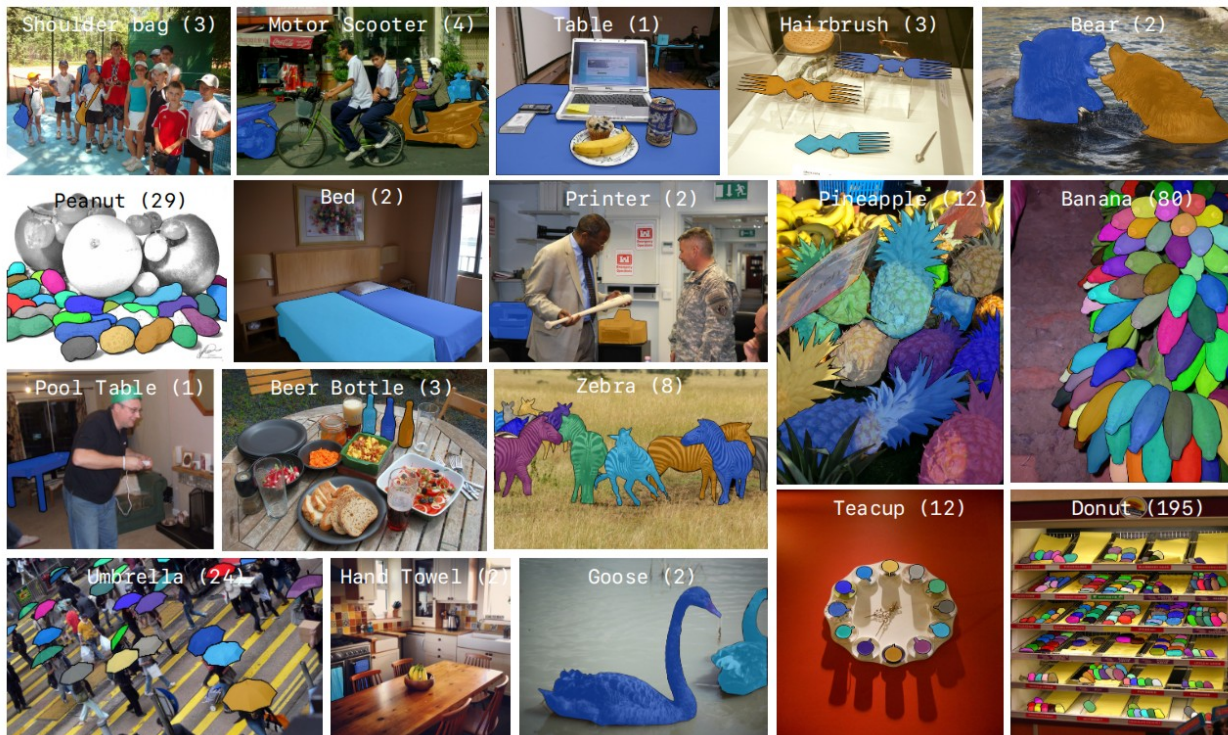
A **52K+** 3D CAD objects

Objaverse/Objaverse-XL



A universe of **10M+** 3D objects

Objaverse-LVIS



Differences between 3D VLMs and 3D VLLMs

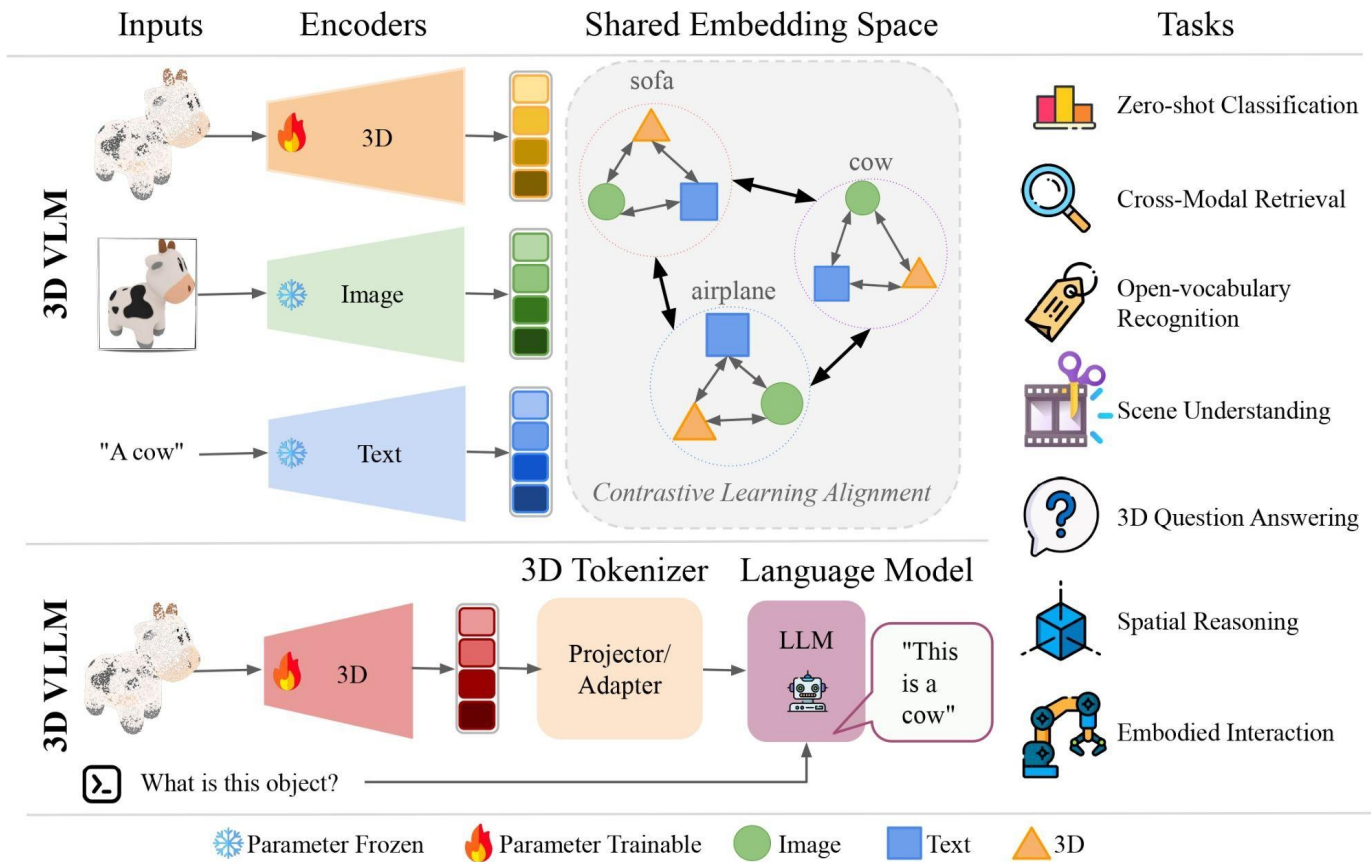
3D VLMs

Can I **connect** this 3D object/scene to language?

3D VLLMs

Can I use this 3D information to **understand** a question, **reason** about it, and **answer** in language?

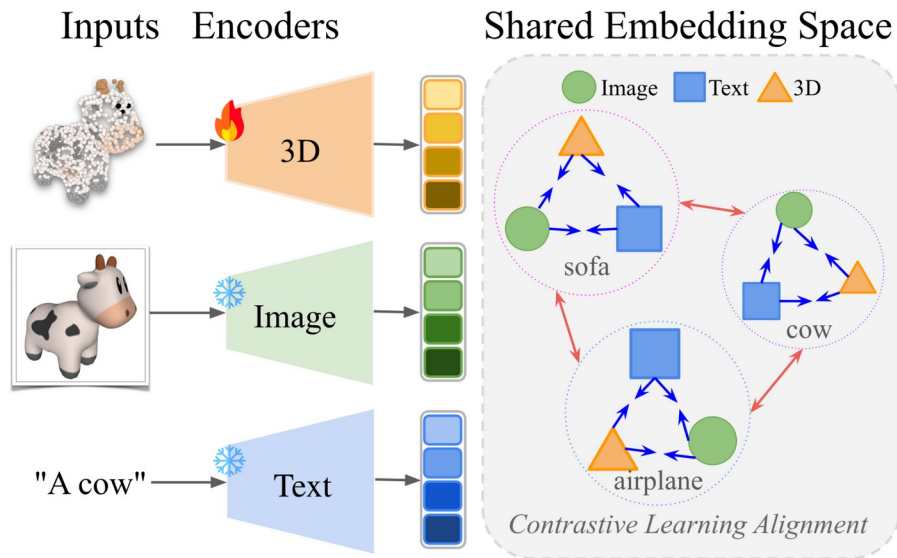
Differences between 3D VLMs and 3D VLLMs



Differences between 3D VLMs and 3D VLLMs

3D VLMs (alignment paradigm)

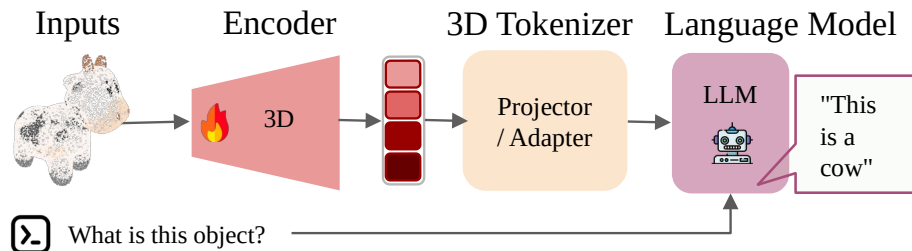
- 3D input, encoder, shared space
- Compare a 3D embedding with image or text embeddings
- Millions of parameters
- **Output:** similarity score or ranking



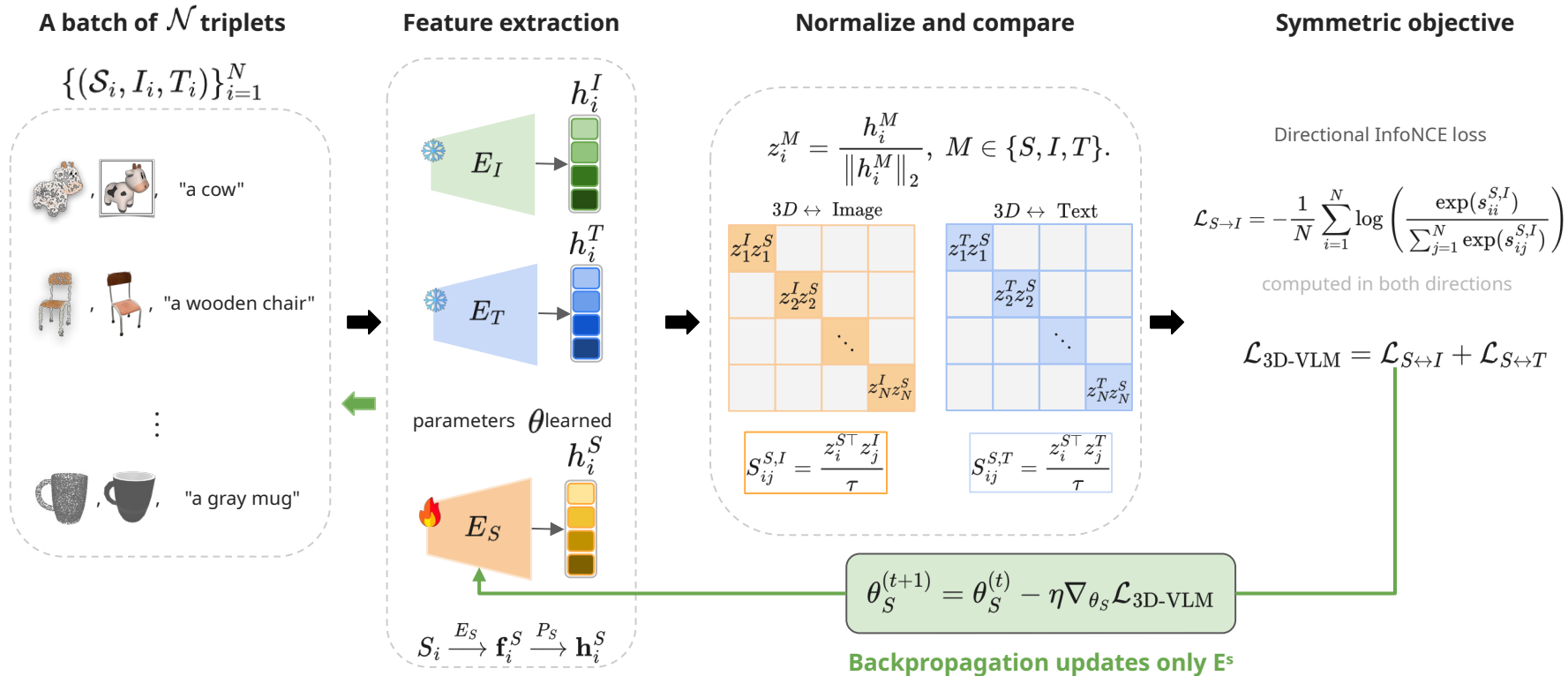
Differences between 3D VLMs and 3D VLLMs

3D VLLMs (generative paradigm)

- 3D input, encoder, projector, Large Language Model (LLM)
- Generate a response conditioned on geometry and a text prompt
- 1 billion to + 13 billions parameters
- **Output:** natural language tokens



Overview of the 3D VLM training pipeline



Outline

- Preliminaries
- Input and encoders
- Geometric view of contrastive alignment
- Contrastive alignment for 3D VLMs
- A taxonomy of 3D VLMs
- Current trends and emerging applications

Input and encoders

Input/Data

- 3D VLMs bridge different structure via $\mathcal{N}$ triplets:

$$\mathcal{D} = \{(S_i, I_i, T_i)\}_{i=1}^N$$

- 3D VLMs uses a dataset of (3D, image, text) pairs:

$$\mathcal{D} = \{ \text{3D cow model}, \text{cow image}, \text{"a cow"} \}$$

Input and encoders

If $\mathcal{D} = \{(S_i, I_i, T_i)\}_{i=1}^N$ is the dataset learn:

1. An **image** encoder: $E_I : \mathbb{R}^{H \times W \times 3} \rightarrow \mathbb{R}^d$

2. A **text** encoder: $E_T : \mathbb{R}^M \rightarrow \mathbb{R}^d$

3. A **3D** encoder: $E_S : \mathbb{R}^{N \times 3} \rightarrow \mathbb{R}^d$

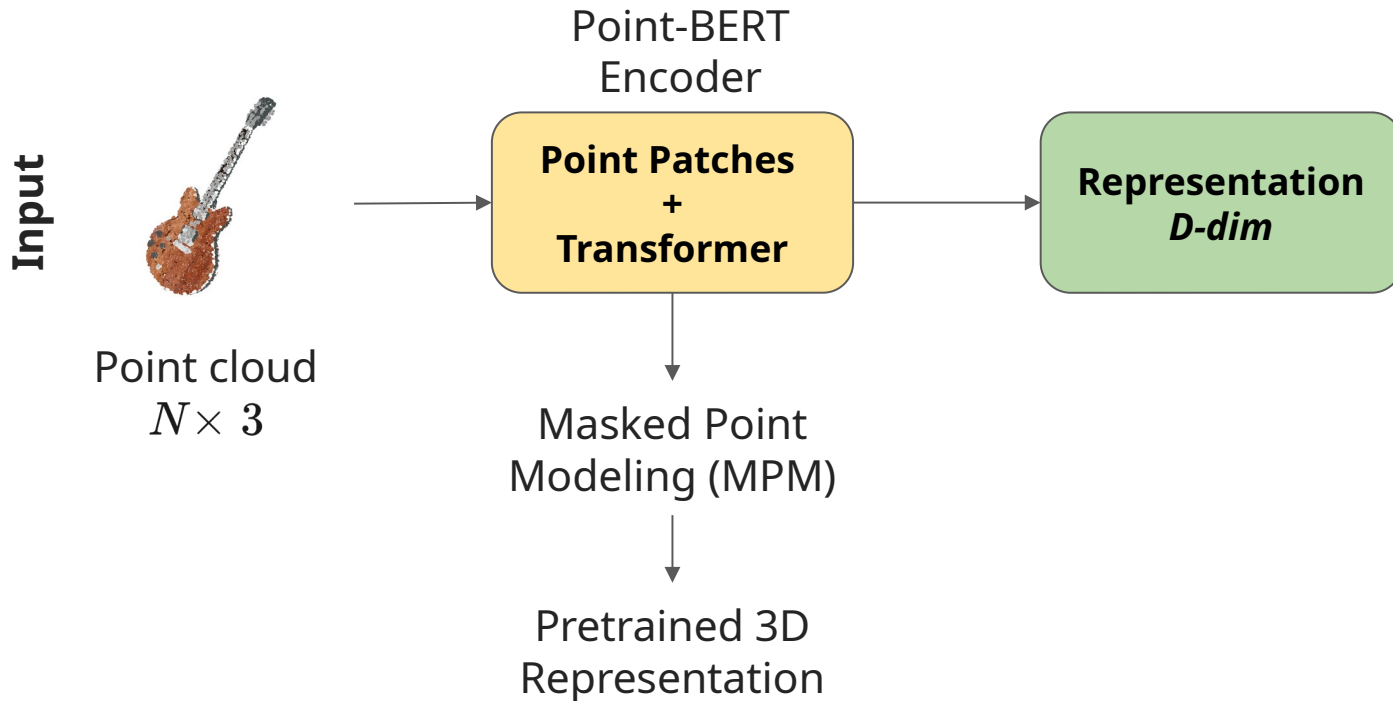
4. Features are mapped by projections layers $\mathcal{P}$ into ℓ_2 normalized embeddings in a common space $\mathbb{R}^d$

Input and encoders

We have some 3D encoder options from this:

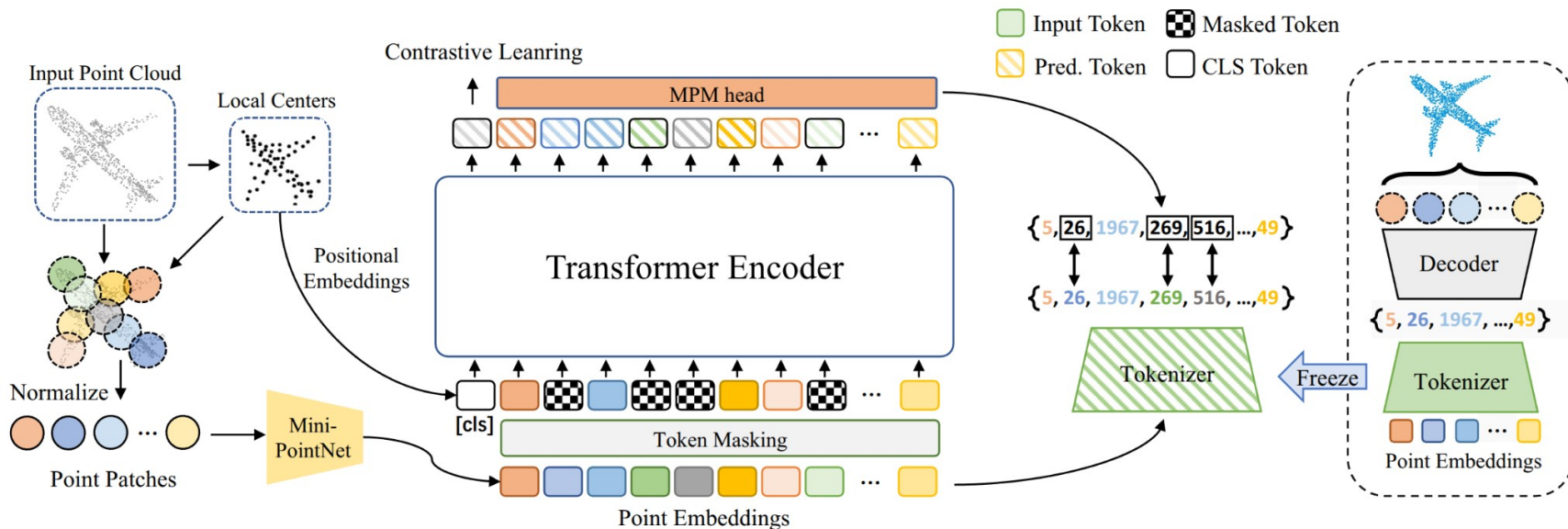
- PointNet / PointNet++
- Point Transformer
- PointMLP
- PointNeXt
- SparseConv
- Point-BERT (we will focus on this encoder)

Point-BERT 3D Encoder Overview



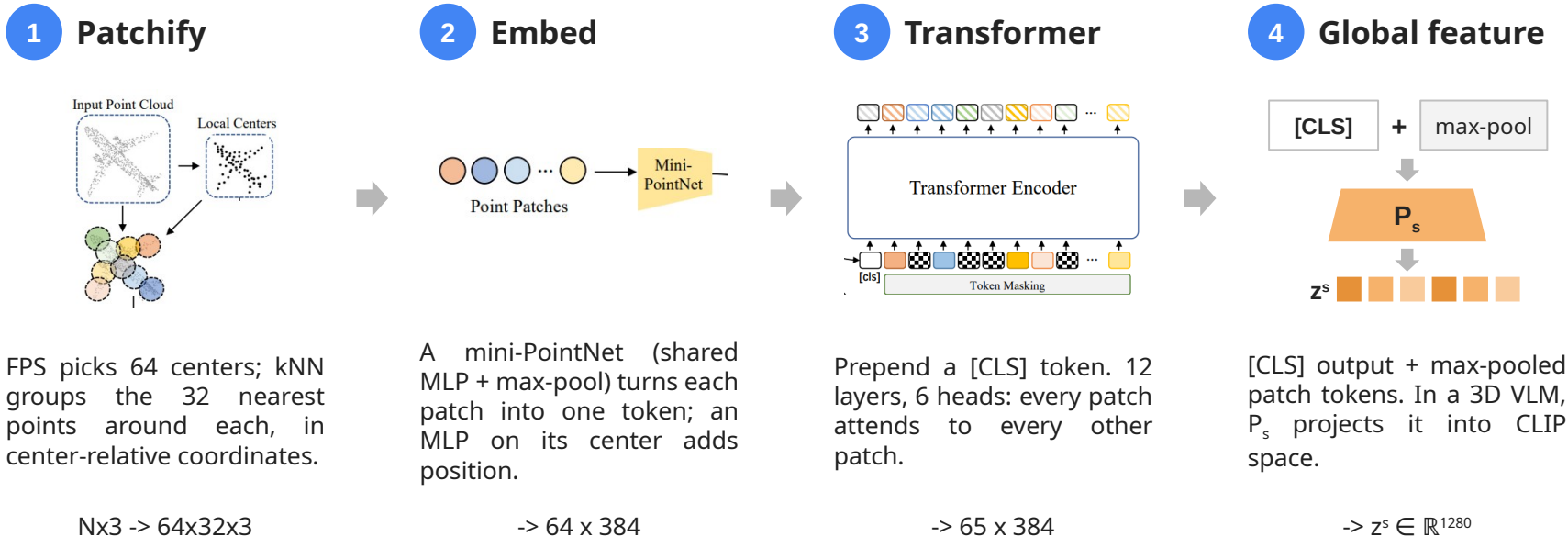
3D Encoder

The pipeline of Point-BERT encoder:



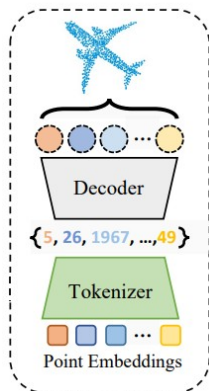
Point-BERT turns point clouds into features

From raw points to one embedding, in four steps:

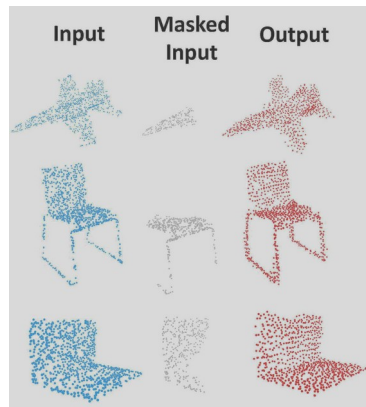


Pre-training Point-BERT: Masked Point Modeling

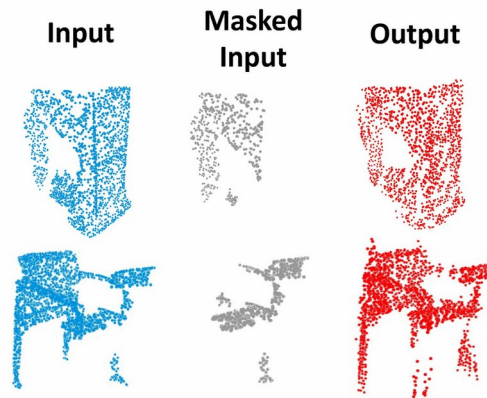
BERT for point clouds: hide patches, then predict what was there



dVAE tokenizer + decoder (pre-training only)



Blue: input **Gray:** visible part
Left: ShapeNet (block masking)



Red: rebuilt from predicted tokens
Right: real ScanObjectNN scans

1. Tokenize: a dVAE (DGCNN encoder) maps each patch to one of 8,192 discrete “point words”



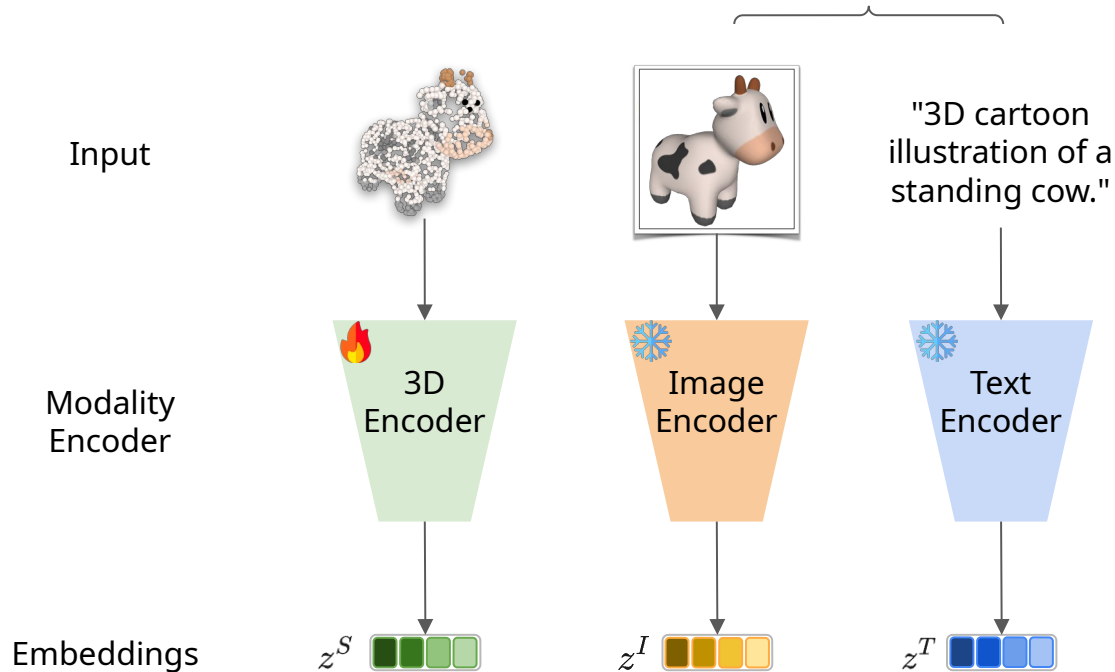
2. Mask: hide 25–45% of the patch embeddings, in contiguous blocks



3. Predict: the Transformer recovers the point-word ID of every masked patch

Three modalities within a single semantic space

OpenCLIP (ViT-bigG-14, laion2b_s39b_b160k)



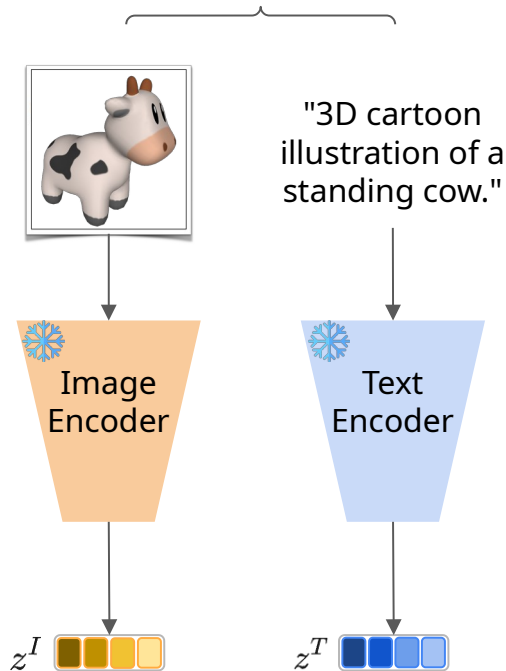
The representations are **completely different**.

But, OpenCLIP has already learned the: *Image and Text*.

Contrastive learning provide a way to make their **embeddings comparable**.

Geometric view of contrastive alignment

OpenCLIP (ViT-bigG-14, laion2b_s39b_b160k)



In this step:

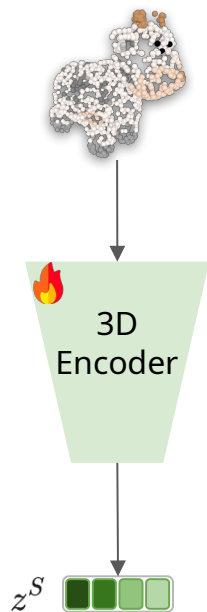
Image and text are **precomputed** features.

$$z^{I,T} \in \mathbb{R}^{1280}$$

Why do this?

Because the OpenCLIP representations are **fixed**. OpenCLIP acts as our **semantic reference space**.

Geometric view of contrastive alignment



The trainable 3D branch starts here:

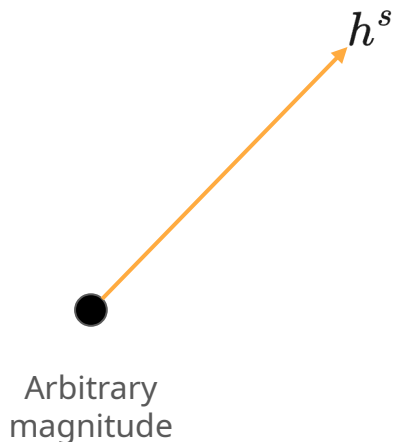
1. Encode the point cloud with Point-BERT;
2. Project the 3D feature into the 1280-dim. OpenCLIP space;
3. L2-normalize the projected embedding.

$$h^s = P_s(E_s(x^s)) \in \mathbb{R}^{1280}$$

Geometric view of contrastive alignment

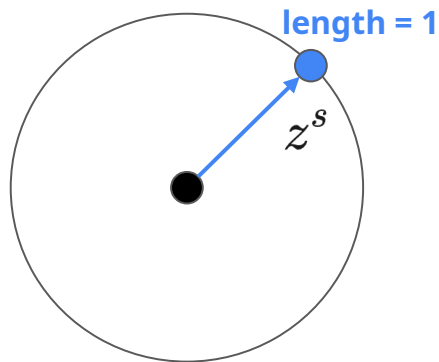
L2-normalization turns embeddings **into directions**

Before normalization



normalize

After normalization



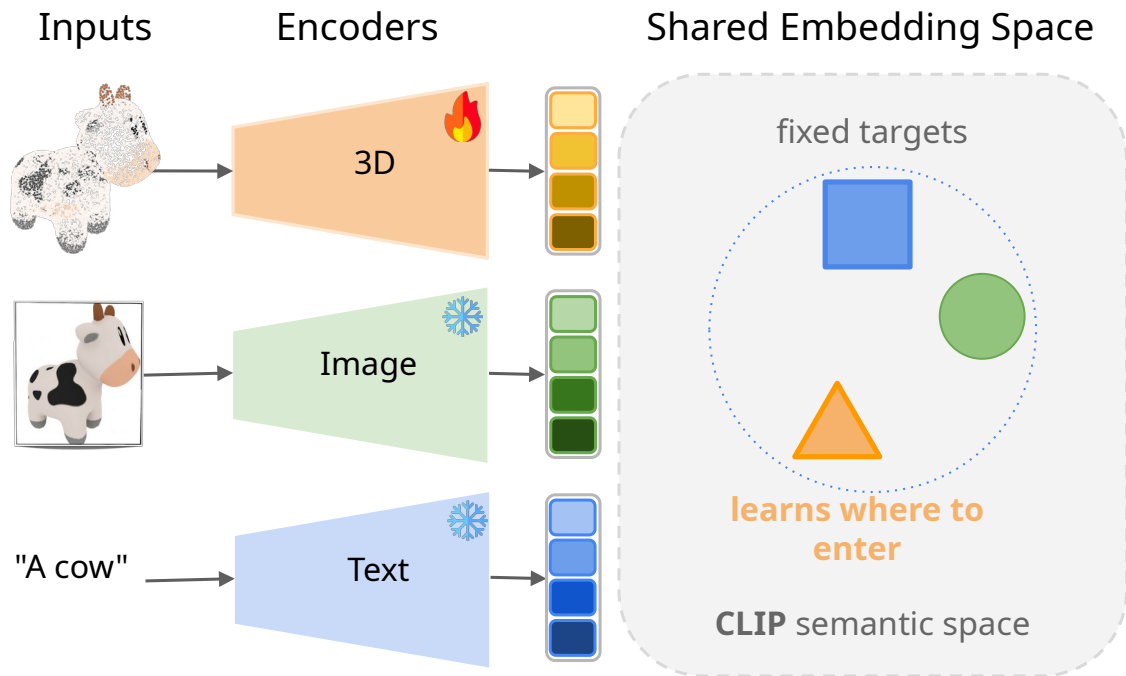
Normalization removes magnitude.
Each embedding becomes a direction.

$$z^M = \frac{h^M}{\|h^M\|_2}$$

$$\|z^M\|_2 = 1$$

$$M \in \{S, I, T\}$$

Geometric view of contrastive alignment



Frozen image and text embeddings **define the semantic reference space.**

The **3D encoder learns to align** its representation through relative similarities with matched and non-matched samples.

$$z^S, z^I, z^T \in \mathbb{S}^{1279}$$

Geometric view of contrastive learning

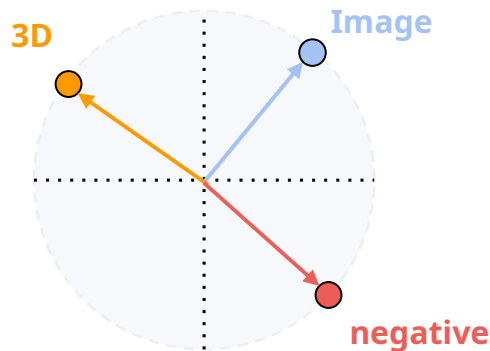
Why a unit hypersphere?

All normalized embeddings lie **on the same geometric surface**.

$$\mathbb{S}^{d-1} = \{z \in \mathbb{R}^d : \|z\|_2 = 1\}$$

For OpenCLIP ViT-bigG/14:

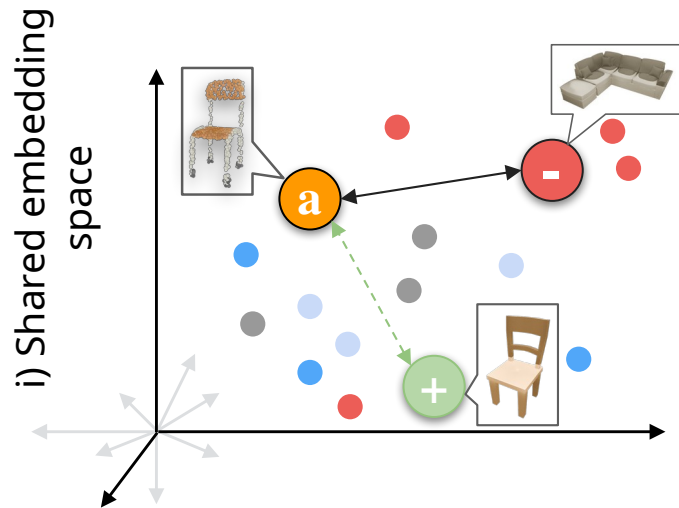
$$z \in \mathbb{R}^{1280} \quad \text{and} \quad \|z\|_2 = 1$$



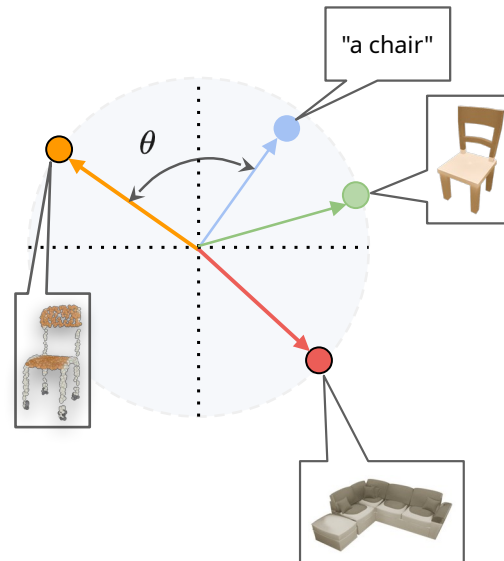
Magnitudes disappears and
directions carries similarity.

Cosine similarity on the unit hypersphere

(a) Before contrastive alignment



ii) 2D slice On the unit hypersphere



$$\theta(a, -) < \theta(a, +)$$

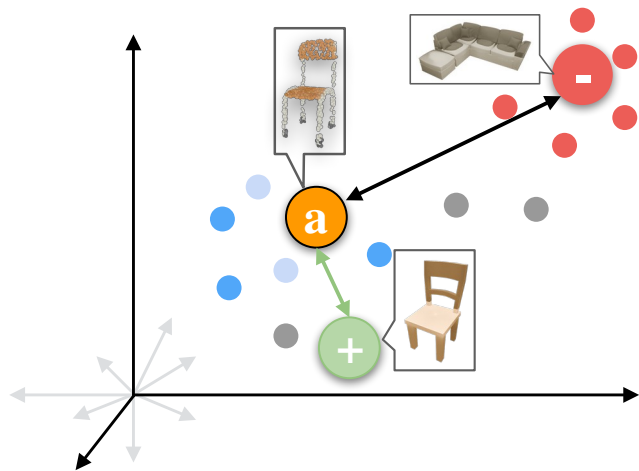
therefore

$$\cos \theta(a, -) > \cos \theta(a, +)$$

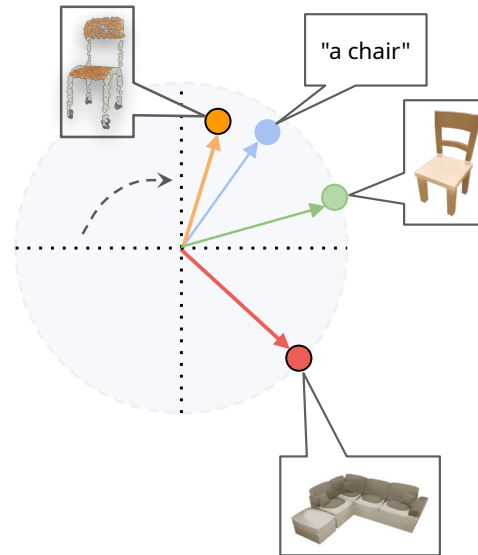
The anchor ranks a negative above its positive.

Cosine similarity on the unit hypersphere

(b) After contrastive alignment



ii) 2D slice On the unit hypersphere



$$\theta(a, +) \downarrow \quad \theta(a, -) \uparrow$$

therefore

$$\cos \theta(a, +) > \cos \theta(a, -)$$

The positive now ranks above the negative.

Contrastive alignment as batch matching

1. Build matches triplets

$$\{(S_i, I_i, T_i)\}_{i=1}^N$$

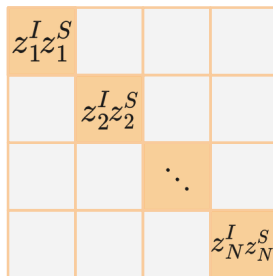


2. Compute all pair scores

Normalized embeddings make the dot product equal cosine similarity

$$S_{ij}^{S,I} = \frac{z_i^{S\top} z_j^I}{\tau}$$

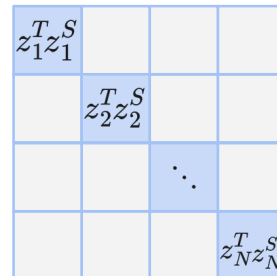
3D $\leftrightarrow$ Image



correct pairs

$$S_{ij}^{S,T} = \frac{z_i^{S\top} z_j^T}{\tau}$$

3D $\leftrightarrow$ Text



in-batch candidates



Diagonal: the matching item



Off-diagonal: alternatives

From normalized embeddings to symmetric InfoNCE loss

The implementation follows the same sequence as the geometric explanation

Pytorch implementation

```
z1 = F.normalize(feet1, dim=1)
z2 = F.normalize(feet2, dim=1)

logits = logit_scale * z1 @ z2.T
labels = torch.arange(logits.shape[0])

loss_12 = F.cross_entropy(logits, labels)
loss_21 = F.cross_entropy(logits.T, labels)
loss = (loss_12 + loss_21) / 2
```

$$z^M = \frac{h^M}{\|h^M\|_2}$$

$$S_{ij}^{S,I} = \frac{z_i^{S\top} z_j^I}{\tau}$$

$$S_{ij}^{S,T} = \frac{z_i^{S\top} z_j^T}{\tau}$$

Labels from similarity matrix

$$\mathcal{L}_{S \leftrightarrow I} = \frac{1}{2} (\mathcal{L}_{S \rightarrow I} + \mathcal{L}_{I \rightarrow S})$$

Symmetric InfoNCE transfers semantics to 3D

3. The loss reward diagonal matches in both retrieval directions

FORWARD DIRECTION

3D $\rightarrow$ image

Which image belongs to this shape?

REVERSE DIRECTION

image $\rightarrow$ 3D

Which shape belongs to this image?

$$\mathcal{L}_{S \rightarrow I} = -\frac{1}{N} \sum_{i=1}^N \log \left(\frac{\exp(s_{ii}^{S,I})}{\sum_{j=1}^N \exp(s_{ij}^{S,I})} \right)$$

$$\mathcal{L}_{S \leftrightarrow I} = \frac{1}{2} (\mathcal{L}_{S \rightarrow I} + \mathcal{L}_{I \rightarrow S})$$

$$\mathcal{L}_{S \leftrightarrow T} = \frac{1}{2} (\mathcal{L}_{S \rightarrow T} + \mathcal{L}_{T \rightarrow S})$$

$$\mathcal{L}_{3D-VLM} = \mathcal{L}_{S \leftrightarrow I} + \mathcal{L}_{S \leftrightarrow T}$$

Contrastive training transfers CLIP's visual-language geometry into the 3D encoder.

Key "ingredients"

What makes contrastive 3D-language alignment work in practice:

1 Paired data at scale

Diverse (3D, image, text) triplets: Objaverse-scale shapes, auto-generated captions (ULIP-2).

2 Data augmentation

Random point dropout, scaling, rotation and jitter; a different rendered view each step; varied caption templates.

3 Large batch size

InfoNCE is an N-way match: more in-batch negatives, more signal per step. OpenCLIP bigG: batch 160k.

4 Hard negative mining

Look-alike negatives (apple vs. cherry) force fine distinctions. Beware false negatives: two chairs in one batch.

5 Temperature τ

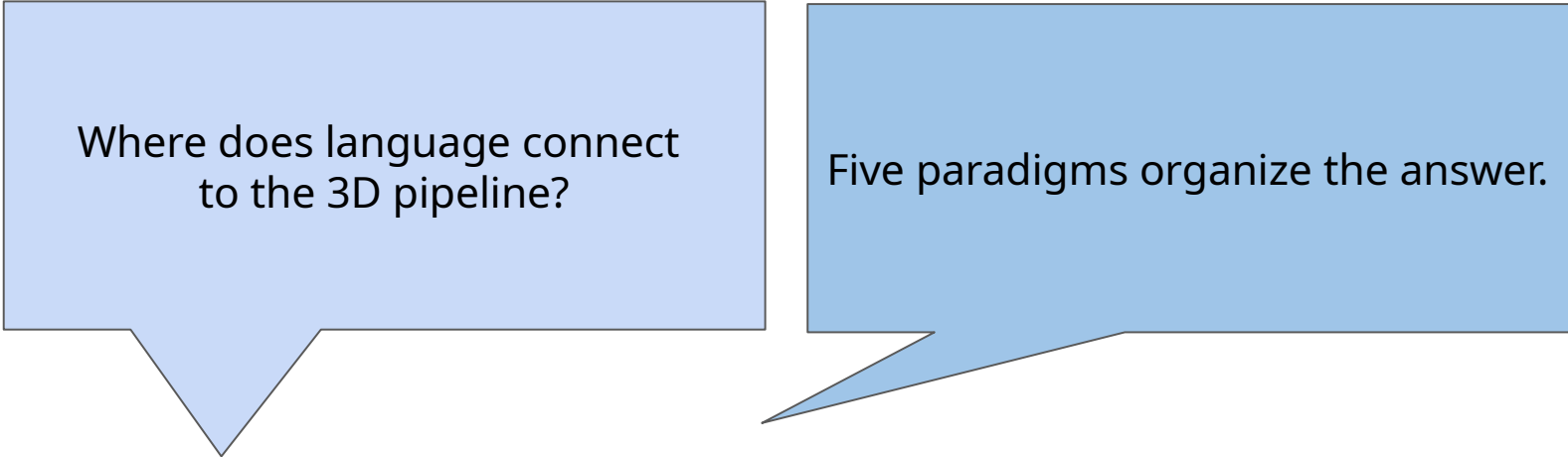
Sets how peaked the softmax over candidates is. Usually a learnable logit scale (CLIP: init 1/0.07, capped at 100).

6 Computational resources

Freeze CLIP and precompute image/text features; mixed precision; gather negatives across GPUs.

From alignment to model families

We have seen how 3D, image, and text embeddings can align.



Where does language connect
to the 3D pipeline?

Five paradigms organize the answer.

A taxonomy of 3D VLMs

From fixed labels -> open vocabulary -> language generation

